



[BIT703 Web Technologies](#) / [Contents](#) / [Topic 11: Server-side web application admin panel development](#) /
[Authorisation and authentication](#)

Additional authentication and authorisation controls

10m



© © Getty Images



[Skip to main content](#)

You may have noticed when working with WordPress in the earlier topics, the users did not simply have user and admin permissions. There is a range of roles, where each role had specific capabilities.

Roles and their capabilities are used to create what is commonly called access control lists (ACLs). This is where users of the web application can only access and perform certain functions in the web application as per the capability of the role. For example, let's look at the following table containing basic roles and capabilities for an e-commerce site.

Role	Capabilities
Administrator	Create, retrieve, update, delete records for: products, orders, customers, site page content
Manager	Create, retrieve, update, delete records for: products, orders, customers
Salesperson	Retrieve, update records for: orders, customers Delete orders.
Customer	Create, retrieve, update, delete records for: Customer own records and orders

Each role can perform CRUD actions based on the level of access for the role, where the customer has the lowest access and the system administrator has the highest access.

The ACL would then link a role to a user within the web application, for example, user Bob could be an admin and user Alice could be a customer. The program code would use the various roles and capabilities to control the data access according to the access level

Skip to main content

In terms of the application you have been working on, you could apply the above table to the users to manage the access control for the dashboard. This would let you enable/disable menu options, prevent certain SQL from being executed and control the page access, routing and redirection. To add the above functionality would require an additional field in the user table that could have one of the following values per user.

- Admin - 1
- Manager - 2
- Salesperson - 3
- Customer - 4

Then the admin, who has the highest access, can allocate roles through the dashboard. When a customer registers for the web application, they would receive the value 4 for their role. Throughout the code, you would have several checks (if . . . else . . .) to see if the logged in user has the correct access for each operation they attempt to perform. For example:



Copy

[Skip to main content](#)

```
1 // for customer and higher access
2 if ($_SESSION['ACL'] < 5 == true) {
3 // do something
4 }
5 // for sales person and higher access
6 if ($_SESSION['ACL'] < 4 == true) {
7 // do something
8 }
9 // for manager and higher access
10 if ($_SESSION['ACL'] < 3 == true) {
11 // do something
12 }
13 // for admin access
14 if ($_SESSION['ACL'] < 2 == true) {
15 // do something
16 }
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)



Thoughts on this page?

